

Performance / Latency and Throughput

1. Overview

Performance is one of the most important **non-functional requirements** in system design.

When users use an application, they care about:

- How fast response comes (**Latency**)
- How many requests system can handle (**Throughput**)
- Whether performance remains stable under load
- Whether system scales as users grow

Examples:

- Opening restaurant menu in food app
- Booking ride
- Submitting payment
- Watching video
- Searching products

2. Definitions

A. Latency

Latency is the total time taken from sending a request until receiving a response.

None

User sends request



Network travel



Server processing



Response travel



User receives response

Total time = Latency

Usually measured in:

- Milliseconds (ms)
- Seconds (s)

B. Throughput

Throughput is the amount of work system can complete in a given time.

Examples:

- Requests per second (RPS)
- Transactions per second (TPS)
- Messages per second
- MB/s processed

3. Latency Components

Total latency is usually:

None

Total Latency =

Client → Server Network Time

+ Queue Wait Time

+ Server Processing Time

+ DB / Cache Calls

+ Response Serialization

+ Server → Client Network Time

4. Real-World Expected Latency

Application	Typical Latency Goal
Web page load	< 2 sec
Search autocomplete	< 100 ms
Payment submit	1–3 sec
Chat send	< 200 ms
Video buffering	Very low

High-frequency trading	Few ms
------------------------	--------

5. Why Low Latency Matters

Slow systems cause:

- User frustration
- Cart abandonment
- Lower engagement
- Revenue loss
- Timeouts
- Poor SEO / ranking

Fast systems improve:

- Retention
- Conversions
- Better UX
- Competitive advantage

6. Throughput Example

Suppose:

None

1 server handles 100 requests/sec

10 servers handle 1000 requests/sec

That is throughput scaling.

7. Latency vs Throughput

They are related but different.

Concept	Meaning
Latency	How fast one request finishes
Throughput	How many requests handled overall

Example:

A system may have:

- Low latency but low throughput
- High throughput but each request slower

Goal depends on use case.

8. Important Requirement Clarification in Interview

Ask:

None

Do users need immediate response?

Can response be async?

Can results arrive later?

Can stale data be shown?

Sometimes acknowledgement is enough.

Example:

None

User uploads file

Immediate response:

Upload accepted

Actual processing later

This reduces latency dramatically.

9. Synchronous vs Asynchronous Response

Synchronous

Wait for full result.

None

Request → Process fully → Return result

Higher latency.

Asynchronous

Return acknowledgment quickly.

None

Request → Accepted

Processing later

Used in:

- Email sending
- Video transcoding
- Report generation
- Notifications

10. Techniques to Reduce Latency

A. Deploy Close to Users

Use geographically nearby data centers.

None

India users → India region

Europe users → EU region

Less network distance.

B. Multi-Region Deployment

If users are global:

Deploy multiple regions.

None

US-East

Europe

Asia

Route users to nearest region.

C. CDN (Content Delivery Network)

Serve static content near users:

- Images
- CSS
- JS
- Videos

Reduces latency.

D. Caching

Use:

- Browser cache
- CDN cache
- Redis cache
- App memory cache

Avoid repeated DB calls.

E. Reduce Payload Size

Smaller responses = faster transmission.

Use:

- Compression
- Pagination
- Select fields only

- Binary protocols

F. Efficient Protocols

Compare:

- REST/JSON (easy, verbose)
- gRPC / Protobuf (compact, fast)
- Custom TCP/UDP protocols (advanced)

G. Optimize Backend Processing

Improve:

- Algorithms
- DB indexes
- Query optimization
- Parallelism
- Efficient code paths

H. Connection Reuse

Avoid new TCP/TLS setup repeatedly.

Use:

- Keep-alive
- HTTP/2
- HTTP/3

I. Batching

Combine multiple operations.

Instead of:

None

100 writes

Do:

None

1 batch write of 100 records

J. Streaming

Send partial data continuously.

Useful for:

- Logs
- Live analytics
- Video
- Chat

11. Read vs Write Optimization

Need to understand workload.

Ask:

None

Read-heavy or Write-heavy?

If Low Read Latency Needed

Precompute data during writes.

Example:

None

Likes count updated on each write

Then reads are instant.

If Writes Need to Be Fast

Store raw events first, aggregate later.

12. Example: View Counter

Question:

Can views be counted after hours?

If YES

Use batch jobs.

None

Collect logs

Aggregate later

If NO (real-time)

Need streaming counters.

None

Update counts continuously

13. Queueing Delay

Even fast servers can be slow when overloaded.

None

Request arrives

Waits in queue 2 sec

Processes in 50 ms

Total latency = 2.05 sec

So queue time matters heavily.

14. Tail Latency (Very Important)

Average latency can hide bad experiences.

Example:

Percentile	Latency
P50	80 ms
P95	700 ms
P99	3 sec

Need to monitor:

- p95
- p99

Because slowest requests hurt users.

15. Throughput Improvement Techniques

- Horizontal scaling
- Load balancing
- Stateless services
- Async queues
- Partitioning
- Caching
- Batch processing
- DB replicas

16. Bottleneck Analysis

System speed limited by slowest component.

Possible bottlenecks:

- CPU
- Memory
- Network
- Disk I/O
- DB locks
- External APIs
- Serialization

17. Formula Insight

Using Little's Law (conceptual):

None

$\text{Throughput} \approx \text{Concurrent Requests} / \text{Latency}$

If latency rises, throughput often drops.

18. Real-World Examples

Food Delivery App

Menu page should load quickly (<2 sec)

Payment App

Confirmation fast but fraud checks may continue async.

YouTube

Video starts quickly via CDN.

Trading App

Milliseconds matter.

19. Trade-offs

Decision	Benefit	Cost
----------	---------	------

Cache	Faster reads	Stale data
Multi-region	Low latency	Complexity
Async processing	Fast response	Eventual consistency
Precompute	Fast reads	Slower writes
Batching	High throughput	Delay

20. Interview Questions to Ask

When discussing performance:

1. Expected QPS / TPS?
2. Peak traffic?
3. Global users?
4. Read/write ratio?
5. Real-time or delayed acceptable?
6. Payload size?
7. SLA latency target?
8. P99 requirement?

21. Interview Answer Example

Q: How would you improve latency of a global application?

Answer:

I'd deploy services in multiple regions close to users, use CDN for static assets, Redis caching for hot reads, optimize DB queries, compress payloads, and route traffic using geo-DNS. I'd also monitor p95/p99 latency.

22. Best Practices

- Measure before optimizing
- Track p50/p95/p99
- Use caches wisely
- Keep services stateless
- Reduce network hops
- Use async where possible
- Scale horizontally
- Load test regularly

23. Short Revision Notes

None

Latency = Time for one request

Throughput = Requests handled per unit time

Improve latency:

- Near users
- CDN
- Cache
- Better queries
- Smaller payloads
- Async flows

Improve throughput:

- Scale out
- Load balance
- Batching
- Queues